



DATA & AI WORKSHOP · PYTHON PHILIPPINES

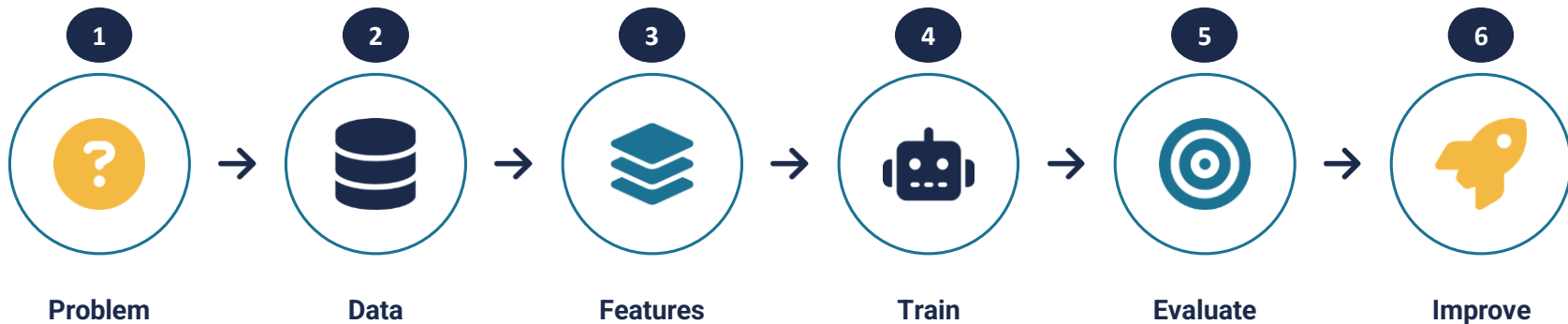
Weather-Weather Lang

What Can Data Tell Us?

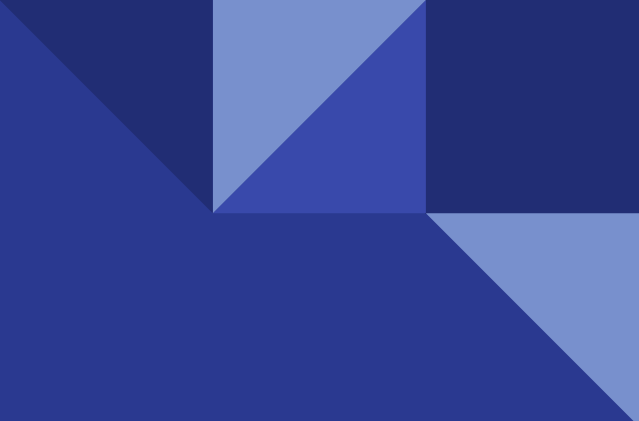
Using real-world weather data to explore classification, from raw data to a working prediction model.

From Raw Data to a Working Prediction

We will follow one continuous workflow — the same one used in real machine learning projects.



By the end, you will have trained and compared three real classification models on real weather data.



Understanding the Problem & the Data

Meet the Australia Weather Data

~99,500 daily weather records across Australian weather stations



Temperature

MinTemp, MaxTemp



Humidity

9am & 3pm readings



Pressure

9am & 3pm readings



Wind

Gust speed & direction



Rainfall

Measured in mm



Rain Today

Yes / No

Classification vs Regression



Regression

Predicts a number.

e.g. "What will tomorrow's temperature be?"



Classification

Predicts a category.

e.g. "Will it rain tomorrow – Yes or No?"

Today's workshop: **we build a classification model.**

Classification, More about the weather data



Classification

Predicts a category:
Will it rain tomorrow —
Yes or No?

Feature → Label

Inputs (Humidity, Pressure...)
→ RainTomorrow

Meet the Dataset

Australia Weather Data · ~99,500 daily records



Temperature



Humidity



Pressure



Wind & Rain

22%

of days have
RainTomorrow = Yes

*This class imbalance matters a lot once we
get to evaluation.*



Data Preparation & Feature Selection

Missing Values: Drop vs. Impute



Drop

Remove rows or columns with missing data.



Simple, no assumptions



You lose real data



Impute

Fill gaps with the mean, median, or mode.



Keeps your dataset intact

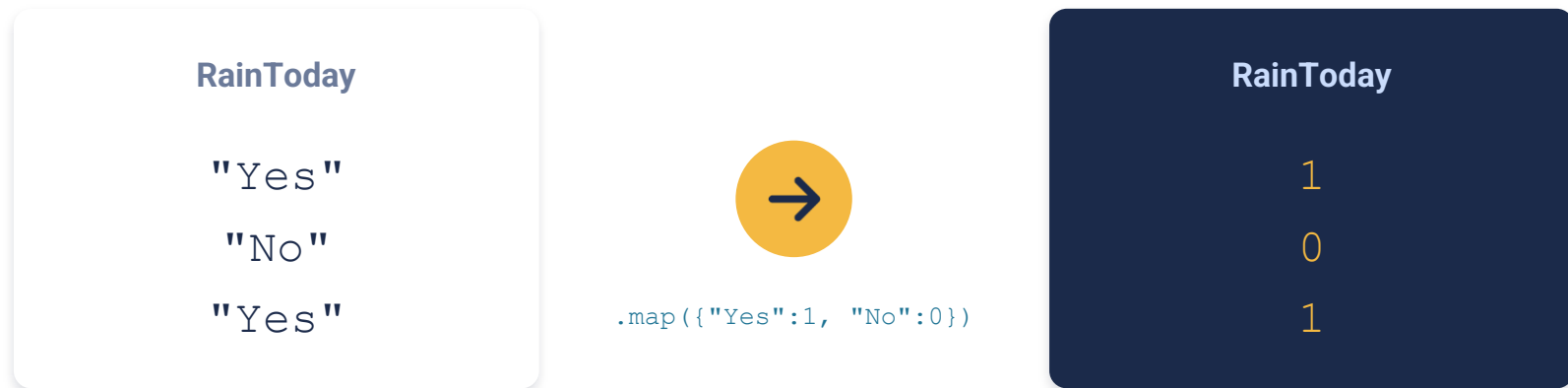


Introduces an assumption

Our approach: drop rows missing the label, impute the rest with column means.

Encoding Categorical Variables

Models only understand numbers — text columns need to be converted first.



This is label encoding — great for simple Yes/No columns. Columns with more categories (like wind direction) typically use one-hot encoding instead, turning each category into its own 0/1 column.

Does Feature Scaling Matter?



Logistic Regression

Sensitive to scale. Needs StandardScaler before training, or large-range features (like Pressure) can dominate small-range ones (like RainToday).



Decision Tree & Random Forest

Scale-invariant. Trees split on thresholds ("Humidity3pm > 70?"), so the raw scale of each feature doesn't affect the result.

This is why only our Logistic Regression pipeline includes a scaling step.

Our Final Feature Set

9 features chosen for relevance and simplicity — enough signal, without the noise.



MinTemp



MaxTemp



Rainfall



Humidity9am



Humidity3pm



Pressure9am



Pressure3pm



WindGustSpeed



RainToday

label → RainTomorrow



Train/Test Split & Model Training

Why We Split Our Data



Training Set (80%)

Like studying with practice questions — the model learns patterns from this data.



Test Set (20%)

Like the real exam — unseen data that tells us how well the model actually generalizes.

```
train_test_split(X, y, test_size=0.2, stratify=y) ← stratify keeps the Rain/No-Rain ratio consistent in both sets
```

Meet the 3 Models



Logistic Regression

Draws a probability boundary between "Rain" and "No Rain".



Decision Tree

Asks a series of yes/no questions to reach a decision.

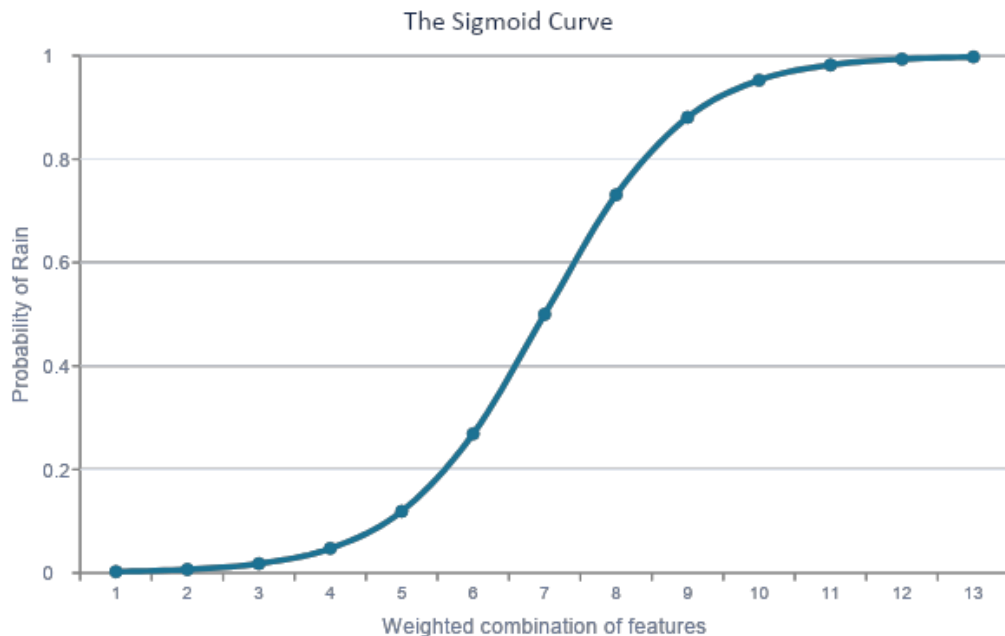


Random Forest

Builds many decision trees and lets them vote on the answer.

We'll train all three on the same data and compare how they perform.

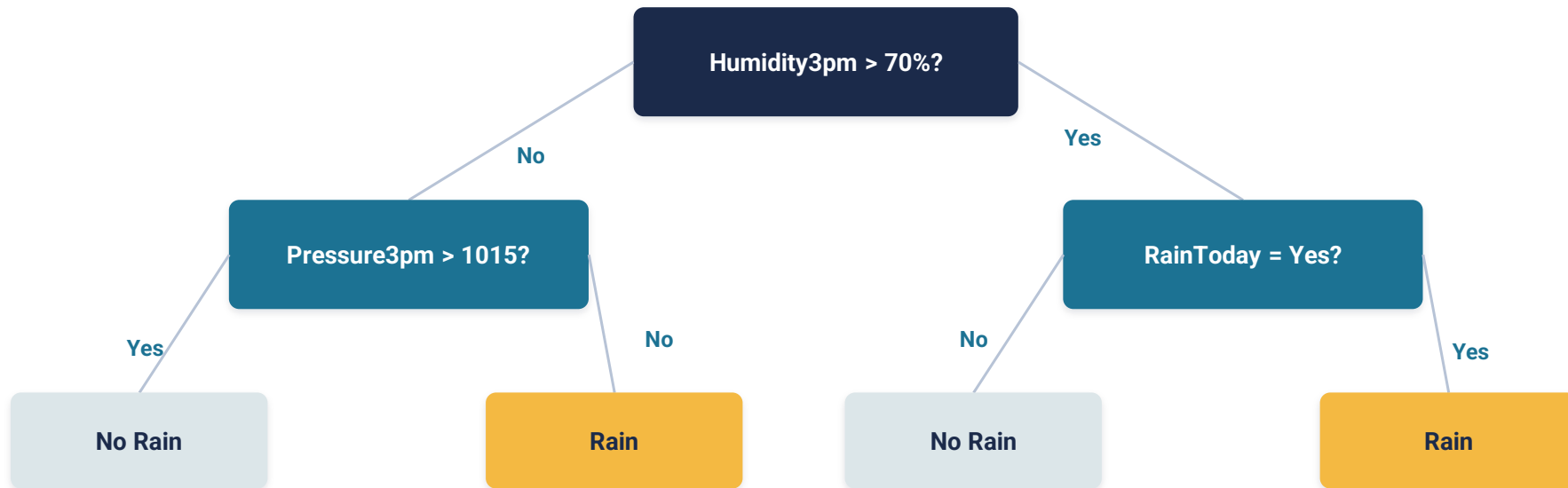
Logistic Regression: Probability Boundaries



How it works

1. Combines all features into a single weighted score.
2. Squashes that score into a probability between 0 and 1.
3. If probability $> 0.5 \rightarrow$ predicts "Rain".

Decision Tree: A Series of Yes/No Questions



This is a simplified view — our actual tree considers all 9 features and goes several levels deeper.

Random Forest: Many Trees, One Vote



Each tree sees a slightly different slice of the data. Averaging their votes smooths out individual mistakes — usually more stable than any single tree.

Evaluation

How do we know if our models
are actually good?



The Confusion Matrix

Example: Logistic Regression on our 19,708 test-set days

	Predicted: No Rain	Predicted: Rain
Actual: No Rain	14,529 True Negative — correctly said no rain	804 False Positive — unnecessary umbrella
Actual: Rain	2,322 False Negative — got caught in the rain	2,053 True Positive — correctly predicted rain

Precision, Recall & F1-Score



Precision

Of the days we predicted rain, how many actually rained?

$$TP / (TP + FP)$$



Recall

Of the days it actually rained, how many did we catch?

$$TP / (TP + FN)$$



F1-Score

A single score balancing precision and recall.

$$2 \times (P \times R) / (P + R)$$

There's usually a trade-off: chasing higher recall (catching more rain days) often costs precision (more false alarms).



Overfitting & Model Improvement

More Complexity Isn't Always Better



The Analogy

An unlimited-depth tree memorizes the training data perfectly (100% train accuracy) – but its test accuracy actually drops to 77.7%.

That's like memorizing answers instead of understanding the material – it works great on questions you've seen, and falls apart on new ones.

Ways to Manage Overfitting



Limit complexity

Cap tree depth, or use fewer/simpler features.



Cross-validation

Test on multiple data splits instead of just one.



Regularization

Penalize overly complex models (e.g. in Logistic Regression).



More / better data

More examples make patterns harder to memorize.

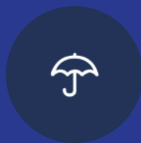
What's Next?

Further improve the modelling
process!

- ✓ Feature Engineering
 - ✓ Hyperparameter tuning
 - ✓ Try other models
 - ✓ Deploy your model
-

Your Challenge: Bring This Model to the Philippines

The workflow you learned today isn't limited to Australia. Can you rebuild it for PH weather?



The Problem

There isn't a ready-made dataset for Philippine weather like the Australia one we used today.



The Solution

Build your own dataset with the Open-Meteo Historical Weather API. It's free, needs no API key or signup, and covers any location worldwide back to 1940.

open-meteo.com/en/docs/historical-weather-api



1. Pick locations

Choose PH cities and get their lat/long coordinates



2. Pull the data

Call the archive API for several years of daily history



3. Engineer the label

Create RainTomorrow from next-day precipitation_sum



4. Reuse the workflow

Same features, same models, same evaluation steps



Let's Talk Data

Thank you! Questions, thoughts, weather predictions of your own?